

第 02 讲

核心概念速查卡：Index / Slice / Reshape / Transpose / Squeeze

用途：写代码或读论文时快速预测 Tensor Shape，避免“代码能跑但结构理解错误”。

核心概念速查卡

1. 先记住一个规则

核心概念

看到任何 Tensor 操作，先写：**操作前 Shape + 轴语义**，再写：**操作后 Shape + 轴语义**。

2. Index 与 Slice

设：

$$X.shape = (B, T, D)$$

代码	输出 Shape	含义
<code>X[0]</code>	<code>(T, D)</code>	固定第 0 个样本，batch 轴消失
<code>X[0:1]</code>	<code>(1, T, D)</code>	只取第 0 个样本，但保留 batch 轴
<code>X[:, 1, :]</code>	<code>(B, D)</code>	所有样本，只取第 1 个 Token
<code>X[:, 1:2, :]</code>	<code>(B, 1, D)</code>	只取第 1 个 Token，但保留 sequence 轴
<code>X[:, -1, :]</code>	<code>(B, D)</code>	所有样本的最后一个 Token

常见误区 / 风险

整数索引通常会删除对应轴；切片通常保留该轴，只改变轴长度。

3. Python 切片语法

```
x[start:stop:step]
```

- start：包含；
- stop：不包含；
- step：步长，默认 1；
- `x[-1]`：最后一个元素；
- `x[:]`：整个轴；
- `x[::2]`：每隔 2 个取一个。

4. Reshape

核心概念

reshape 改变 Shape，但元素总数必须保持不变。

例如：

$$(24,) \rightarrow (2, 3, 4) \rightarrow (6, 4)$$

都合法，因为 size 都是 24。

基本关系：

$$\prod \text{old shape} = \prod \text{new shape}$$

```
x.reshape(2, 3, 4)
x.reshape(2, 3, -1) # 自动推断最后一维
```

5. Transpose

transpose 改变轴的顺序。

```
X.transpose(0, 2, 1)
```

如果：

$$X.shape = (2, 3, 4)$$

则：

$$(2, 3, 4) \rightarrow (2, 4, 3)$$

轴标签如果原来是：

$$[B, T, D]$$

现在变成：

$$[B, D, T]$$

6. Reshape 与 Transpose 的区别

操作	改变什么	元素总数
reshape	如何把连续元素分组为不同 Shape	不变
transpose	轴的排列顺序	不变

同样从 (2, 3) 到 (3, 2)，reshape 与 transpose 得到的数值布局通常不同。

7. 增加与删除长度为 1 的轴

NumPy：

```
np.expand_dims(x, axis=0)
np.squeeze(x, axis=0)
```

PyTorch 以后对应：

```
x.unsqueeze(0)
x.squeeze(0)
```

典型变化：

$$(4,) \rightarrow (1, 4)$$

用于给单样本增加 batch 轴。

8. LLM 中最值得记住的两个 Shape 操作

- 最后一个 Token: `logits[:, -1, :]`, 从 (B, T, V) 到 (B, V) 。
- Multi-Head Attention 常见: $(B, T, H, D_h) \rightarrow (B, H, T, D_h)$, 本质上是交换轴。

9. Shape 调试四步法

1. 写出当前 Shape;
2. 给每个轴贴标签, 例如 B/T/D;
3. 判断当前代码对哪个轴做了什么;
4. 写出操作后 Shape, 再运行验证。

常见风险

- `squeeze()` 无参数可能误删 `batch size = 1` 的轴;
- $(D,)$ 、 $(1, D)$ 、 $(D, 1)$ 不是一回事;
- Shape 一样不代表语义一样;
- 代码没报错不代表轴顺序一定正确。